MLOPS | ASSIGNMENT 01

Heart Disease Risk Classification

An end-to-end, reproducible MLOps implementation

Course	Student	Submission date
MLOps(AIMLCZG523)	JAGANNATH – 2025AE05167	16-10-2026

Executive summary

This project builds and evaluates a binary classifier using the processed Cleveland subset of the UCI Heart Disease dataset. It packages data loading, missing-value handling, encoding, scaling, model selection, evaluation, and inference as a reproducible workflow. Logistic Regression and Random Forest are tuned and compared; the selected Logistic Regression pipeline is served through FastAPI, containerized with Docker, tracked with MLflow, and configured for Prometheus/Grafana and Kubernetes.

Repository

GitHub: https://github.com/Jagguchagi/Heart_Disease_Prediction

1. Objective, scope, and architecture

The assignment asks for dataset acquisition and EDA; feature engineering and model comparison; experiment tracking; a reusable model and environment; automated testing and CI; Dockerized inference; a production-style deployment; monitoring and logging; and a report with reproducible access instructions. The implementation addresses those requirements in one small, local-first pipeline.

Stage	Implementation	Output
Acquire and inspect	UCI Cleveland source file; validated loader	303 rows, 13 features, binary target
Explore and prepare	EDA plots; train-fold imputers, scaling, encoding	Summary JSON, plots, fitted preprocessing
Train and track	Tuned Logistic Regression and Random Forest; MLflow	Metrics, plots, model artifacts
Serve and observe	FastAPI; request metrics; Prometheus/Grafana	Validated JSON prediction and dashboards
Package and deploy	Docker/Compose; Minikube Deployment and Service	Containerized API and local LoadBalancer

System flow

UCI data -> loader and EDA -> split and preprocessing pipeline -> tuning / evaluation / MLflow -> selected joblib model -> FastAPI -> Docker -> Kubernetes Service
|-> Prometheus metrics -> Grafana dashboard

Scope and interpretation

The target is formed by mapping source diagnosis `num=0` to class 0 and `num=1..4` to class 1. The term risk in the project title describes the educational binary classification task; model probabilities are not calibrated clinical risk estimates. All performance figures below apply only to the stated split and dataset.

2. Data source, acquisition, and quality

The data is the processed Cleveland subset from the UCI Machine Learning Repository Heart Disease collection [1]. The checked-in source, `data/heart+disease/processed.cleveland.data`, is headerless and comma-separated. `heart-disease.names` documents the variables and the original ordinal diagnosis. The included acquisition script documents how to retrieve the archive when the local file is absent.

The loader assigns the 13 documented predictor names plus the raw `num` label, treats `?` and empty strings as missing, rejects absent/invalid target labels, and creates `target = (num > 0)`. It keeps the target out of model features.

Data check	Observed result	Handling / meaning
Rows and columns	303 rows; 13 inputs + target	Processed Cleveland subset
Missing input cells	6 total: `ca` 4; `thal` 2	Imputed within fitted pipeline
Duplicate rows	0	No duplicate rows in inspected file
Target 0	164 (54.13%)	Source diagnosis is zero
Target 1	139 (45.87%)	Any source diagnosis 1 through 4

Feature groups

Continuous measurements (5)	Coded categorical inputs (8)
age, trestbps, chol, thalach, oldpeak	sex, cp, fbs, restecg, exang, slope, ca, thal

The classes are reasonably close in size but are not exactly balanced. Stratification is used in the train/test split and folds to retain similar class proportions. These checks are repeated in `src/data_preparation.py` and covered by tests.

Reproduction

From the repository root: `python -m src.data_preparation`, `python -m src.eda`, and `python -m src.train`. Requirements are pinned in `requirements.txt`; the serving image uses the smaller `requirements-runtime.txt`.

3. Exploratory data analysis: balance and association

EDA was run before model fitting. The class-count chart makes the mild class imbalance visible. Continuous-variable correlations are shown separately: coded category values are not treated as measurements with meaningful numeric distances.

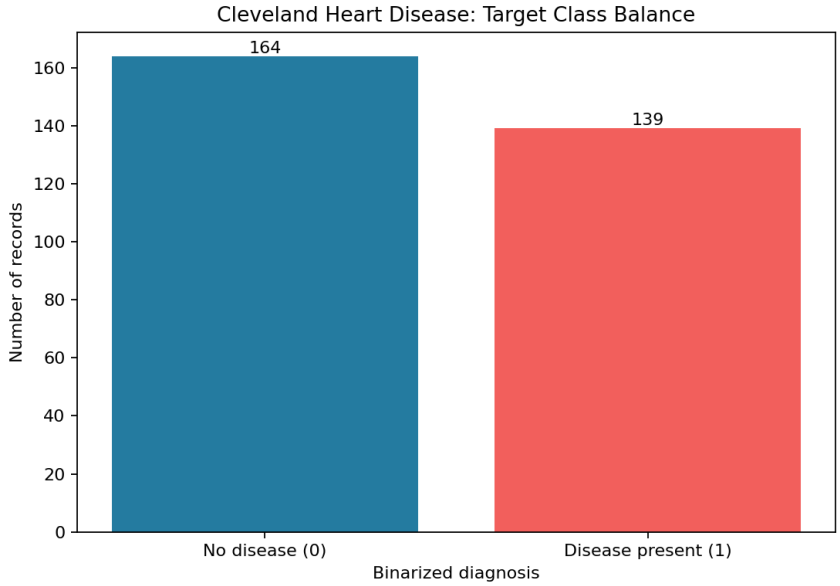


Figure 1. Binary class counts: 164 no-disease and 139 disease-present observations.

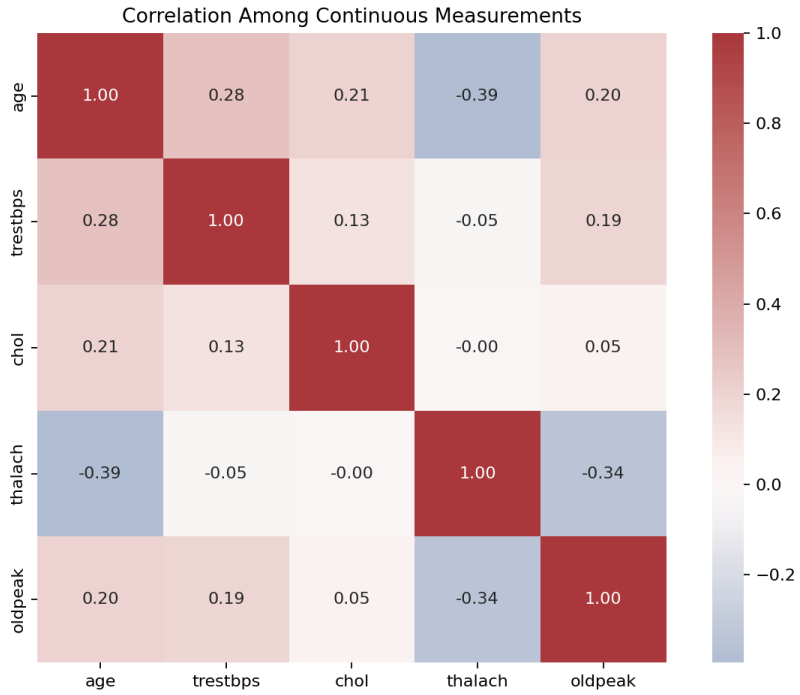


Figure 2. Pearson correlations among the five continuous measurements only.

Observed patterns

- The class-count difference is modest (25 records), so accuracy is interpretable alongside class-specific recall and precision.
- Among the displayed continuous measurements, age and maximum heart rate (`thalach`) have a moderate negative correlation (about -0.39); `thalach` and `oldpeak` are also moderately negatively correlated (about -0.34).
- Other displayed pairwise correlations are smaller in magnitude. Correlation is descriptive and does not establish causation or a physiological mechanism.

The descriptive numbers are stored in `artifacts/eda_summary.json`; the chart is generated by `src/eda.py` and can be regenerated from the included data.

4. Exploratory data analysis: feature distributions

The following overlaid histograms compare the distributions of the continuous inputs between target classes. Overlap is substantial for several measurements, which is expected in a small observational dataset and cautions against interpreting any single feature as a deterministic separator.

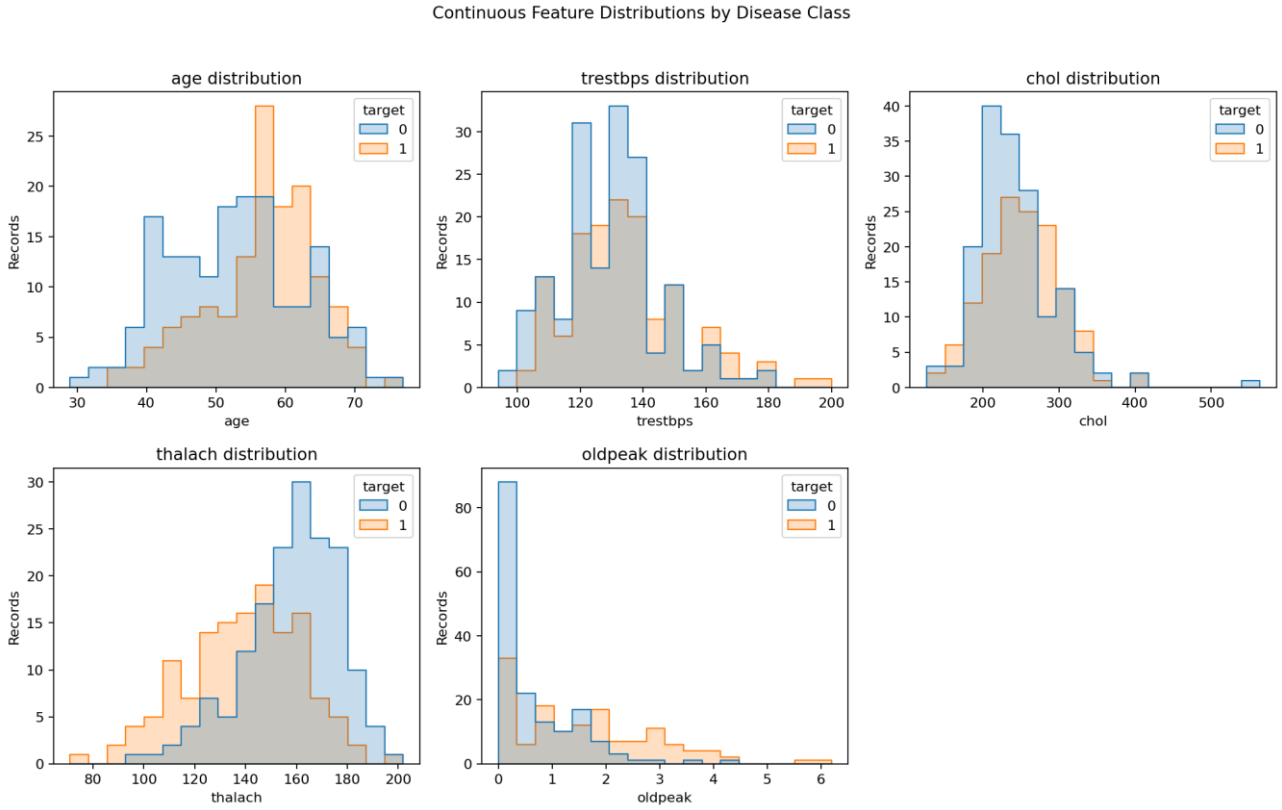


Figure 3. Continuous-feature histograms grouped by the binarized target.

Interpretation and limitations of EDA

- The plotted age distributions overlap; their shapes alone do not provide a reliable classifier.
- The disease-present group contains more observations in lower `thalach` ranges in this sample, while values still overlap across groups.
- `oldpeak` is concentrated near zero in both groups and has a right tail. The distributions are affected by the selected sample and target conversion.
- These are unadjusted descriptive comparisons. They do not control for other variables, establish causality, or prove generalization to a new population.

The source data has a small number of missing predictor values, concentrated in `ca` and `thal`. Those values are not filled globally during EDA/model selection; imputation is fitted within each training fold to avoid leakage.

5. Feature engineering and model development

The target is separated before splitting. A fixed 80/20 stratified split (random seed 42) yields 242 training and 61 test records. The held-out test set is reserved for the final evaluation and is not used to choose hyperparameters.

Input type	Pipeline transformation	Reason
Continuous: age, trestbps, chol, thalach, oldpeak	Median imputation -> StandardScaler	Robust simple missing-value handling; comparable scale for Logistic Regression
Categorical codes: sex, cp, fbs, restecg, exang, slope, ca, thal	Most-frequent imputation -> one-hot encoding (`handle_unknown=ignore`)	Avoid treating category codes as ordinal distances; tolerate unseen categories

Training and selection protocol

- Both estimators are complete scikit-learn Pipelines, so preprocessing is refit inside cross-validation and inference uses the same transformations.
- GridSearchCV tunes on the training partition using 3-fold stratified ROC-AUC. Logistic Regression searches C in {0.1, 1, 10}; Random Forest searches estimator count {100, 250}, depth {None, 5}, and minimum leaf size {1, 3}.
- The best configuration for each model is compared on the training portion with 5-fold stratified cross-validation for accuracy, precision, recall, and ROC-AUC.
- The model with the highest mean 5-fold ROC-AUC is selected, then evaluated on the held-out test set. Seed 42 makes the split and model initialization reproducible.

The trained preprocessing-plus-classifier pipeline is saved as a single `joblib` artifact. This avoids a separate, error-prone inference preprocessing implementation.

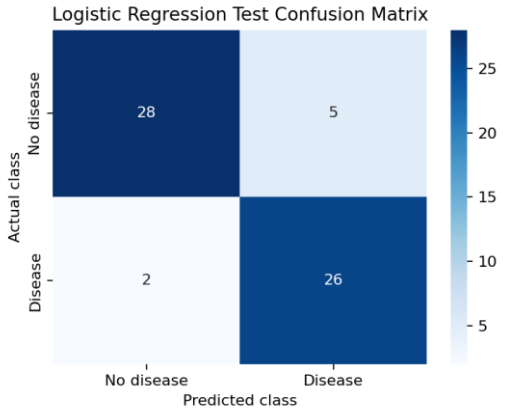
6. Evaluation results and model choice

Model	CV acc / prec / recall / AUC	Test acc / prec / recall / AUC	Selected parameters
Logistic Regression	0.839 / 0.868 / 0.774 / 0.902	0.885 / 0.839 / 0.929 / 0.965	{"classifier__C": 0.1}
Random Forest	0.814 / 0.800 / 0.801 / 0.899	0.869 / 0.812 / 0.929 / 0.951	{"classifier__max_depth": null, "classifier__min_samples_leaf": 3, "classifier__n_estimators": 250}

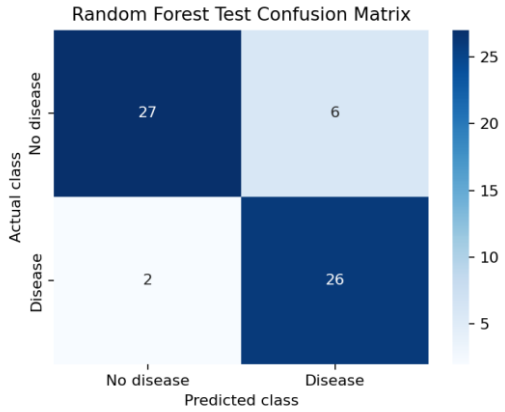
Logistic Regression was selected: its mean 5-fold training ROC-AUC was 0.902, compared with 0.899 for Random Forest. On the held-out test split, Logistic Regression achieved ROC-AUC 0.965, accuracy 0.885, precision 0.839, and recall 0.929. The difference in cross-validation ROC-AUC is small; it is not evidence that Logistic Regression will outperform in every population or split.

Held-out confusion matrices

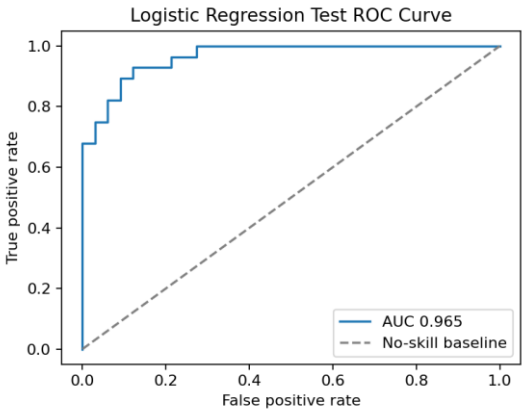
Accuracy is the share of all test cases classified correctly; precision is the share of predicted positives that are positive; recall is the share of actual positive cases detected; ROC-AUC summarizes ranking across thresholds. These metrics describe this split only.



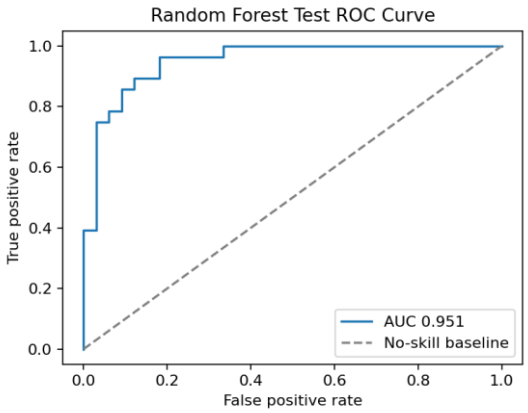
Logistic Regression confusion matrix



Random Forest confusion matrix



Logistic Regression ROC curve (AUC 0.965)



Random Forest ROC curve (AUC 0.951)

Each model identifies 26 of 28 disease-present test observations (2 false negatives). Logistic Regression has 28 true negatives and 5 false positives; Random Forest has 27 true negatives and 6 false positives. In screening contexts, false negatives merit attention, but recall alone does not establish clinical value.

The corresponding ROC curves and all metrics are retained in `artifacts/evaluation/` and `artifacts/model_comparison.csv`.

7. Experiment tracking, packaging, and API

Training creates or reuses the `heart-disease-cleveland` MLflow experiment backed by SQLite. Each model run logs its model and tuning parameters, selection protocol, CV and test metrics, evaluation plots, EDA outputs, and fitted estimator artifact. The bundled local store (`Project/mlflow.db` with `Project/mlruns/`) was regenerated together so the local artifact paths match.

Run	Best parameters	CV ROC-AUC	Test ROC-AUC	Tracked fields
Logistic Regression	C=0.1	0.902	0.965	9 metrics; 6 parameters; plots, EDA and model
Random Forest	250 trees; depth=None; min leaf=3	0.899	0.951	9 metrics; 8 parameters; plots, EDA and model

Reusable model and serving contract

- The selected full pipeline is `models/heart_disease_pipeline.joblib`; `models/model_metadata.json` records selection, features, parameters, and final metrics.
- FastAPI exposes `GET /health`, `POST /predict`, and `GET /metrics`. Pydantic validates 13 feature fields, rejects extra fields, and allows null `ca` and `thal` values for pipeline imputation.
- Prediction responses contain the binary class, predicted-class confidence, and class-1 disease probability. Request logs record outcomes without logging feature payloads.
- The image uses runtime-only dependencies, Python 3.10 slim, and a non-root application user. `requirements.txt` and `requirements-runtime.txt` separate training and serving dependencies.

Sample API response

{"prediction": 0, "confidence": 0.5762, "disease_probability": 0.4238}

8. CI/CD, Docker, and monitoring

Continuous integration

GitHub Actions workflow `.github/workflows/ci.yml` runs on pushes to `main`, pull requests, or manual dispatch. The publicly visible run completed successfully (run #3, commit `a55ef8e`). It installs dependencies, runs Ruff and pytest, regenerates EDA and model runs, builds the API image, checks health/prediction/metrics endpoints, and uploads evidence. The archived JUnit result contains 8 tests, 0 failures, and 0 errors.

Container and Compose verification

On Rocky Linux 9.5, Docker Engine 28.0.1 and Compose 2.33.1 were available. Compose reported the API, Prometheus, and Grafana services as running. `/health` returned `status=ok` and `model_ready=true`; `/predict` returned the expected JSON contract. The Compose stack is a local demonstration, not a publicly exposed service.

Monitoring and observability

The API exports Prometheus metrics through `/metrics`. Prometheus scrapes the API and Grafana provisions a Prometheus datasource and an API dashboard with request-rate, 95th-percentile latency, and 5xx panels. The demonstrated dashboard showed populated request and latency series; the 5xx panel was blank because no server-error series was recorded in the displayed interval. Request feature values are intentionally omitted from application logs.

9. Kubernetes deployment, limitations, and conclusion

Local Kubernetes deployment

The Rocky Linux 9.5 lab VM ran Minikube 1.35.0 with the Docker driver and Kubernetes 1.32.0. The Docker image was loaded into the cluster, and `deployment/deployment.yaml` created a Deployment and LoadBalancer Service. Rollout succeeded with one Ready replica, zero restarts, and service external IP `10.105.118.39` while the Minikube tunnel was active. Requests through the Kubernetes LoadBalancer returned `model_ready=true` and a valid prediction response. This address is VM/cluster-local for the lab, not a public cloud endpoint.

The manifest defines readiness and liveness probes against `/health`, a named HTTP port, one replica, and CPU/memory requests and limits. For another cluster, the image must be made available to that cluster and its LoadBalancer/Ingress implementation must be configured.

Limitations and responsible use

- The dataset has only 303 historical Cleveland observations. The split-based estimates are uncertain and do not establish performance on other hospitals, populations, or current clinical practice.
- Binarizing `num` merges severities 1 through 4 and loses ordinal detail. Threshold choice changes false-positive/false-negative tradeoffs.
- No external validation, prospective evaluation, calibration study, subgroup fairness audit, clinical review, or regulatory review is claimed.
- The API is an educational demo. Do not use it to diagnose, triage, treat, or make decisions about real people; do not send identifiable health data.

Conclusion

The implementation demonstrates a traceable path from source data through leakage-aware training, model comparison and MLflow tracking to a tested API, container, monitoring stack, and local Kubernetes endpoint. The final selected pipeline is reproducible from the supplied requirements and data. Add author details, personal EDA commentary, CI/Docker/Prometheus screenshots, final formatting, and a short demonstration recording before submission.

References

[1] UCI Machine Learning Repository, Heart Disease dataset (processed Cleveland subset), https://archive.ics.uci.edu/dataset/45/heart+disease. Dataset metadata credits Andras Janosi, William Steinbrunn, Matthias Pfisterer, and Robert Detrano.

[2] scikit-learn documentation, Pipeline, preprocessing, model selection, and metrics, https://scikit-learn.org/stable/.

[3] MLflow Tracking documentation, https://mlflow.org/docs/latest/ml/tracking/.

[4] FastAPI documentation, https://fastapi.tiangolo.com/; Docker documentation, https://docs.docker.com/.

[5] Kubernetes documentation, https://kubernetes.io/docs/; Prometheus documentation, https://prometheus.io/docs/; Grafana documentation, https://grafana.com/docs/.

Appendix A. Infrastructure evidence

The following captures document the monitoring dashboard and the successful Minikube LoadBalancer test on the Rocky Linux lab VM. They supplement the machine-readable metrics, configuration, and model evaluation artifacts in this bundle.

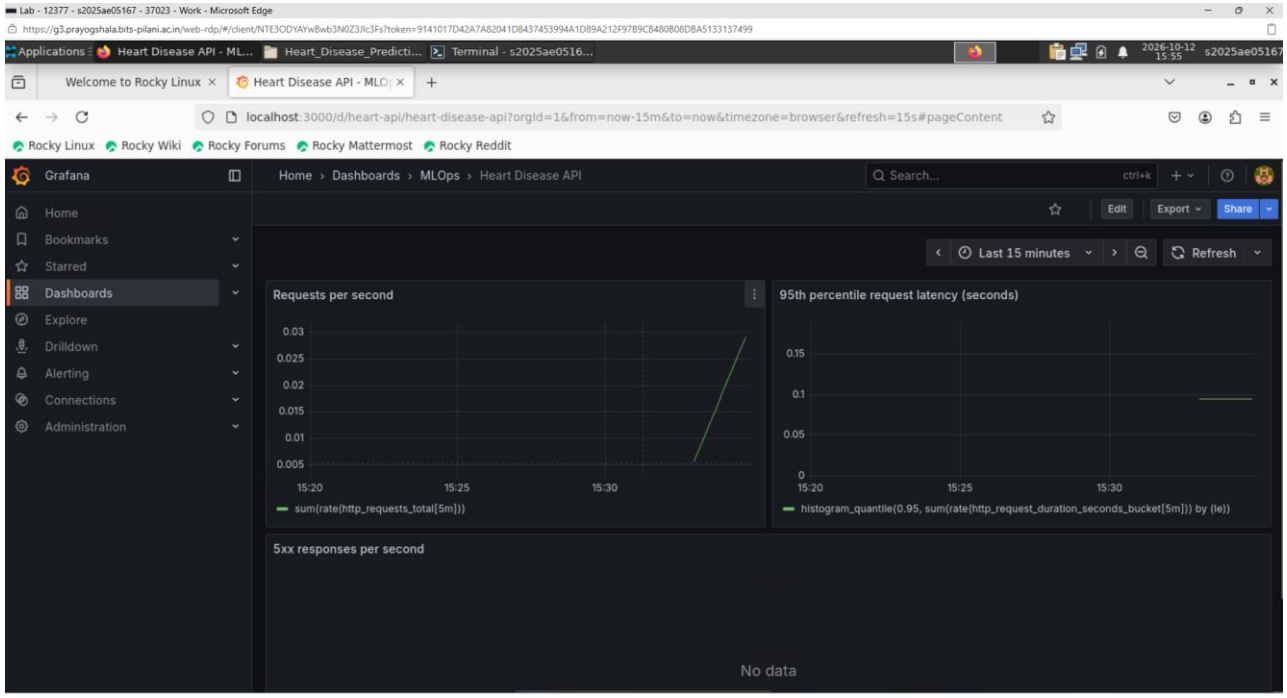


Figure A1. Grafana Heart Disease API dashboard; request rate and p95 latency are populated for the last 15 minutes. No 5xx series was recorded in this interval.

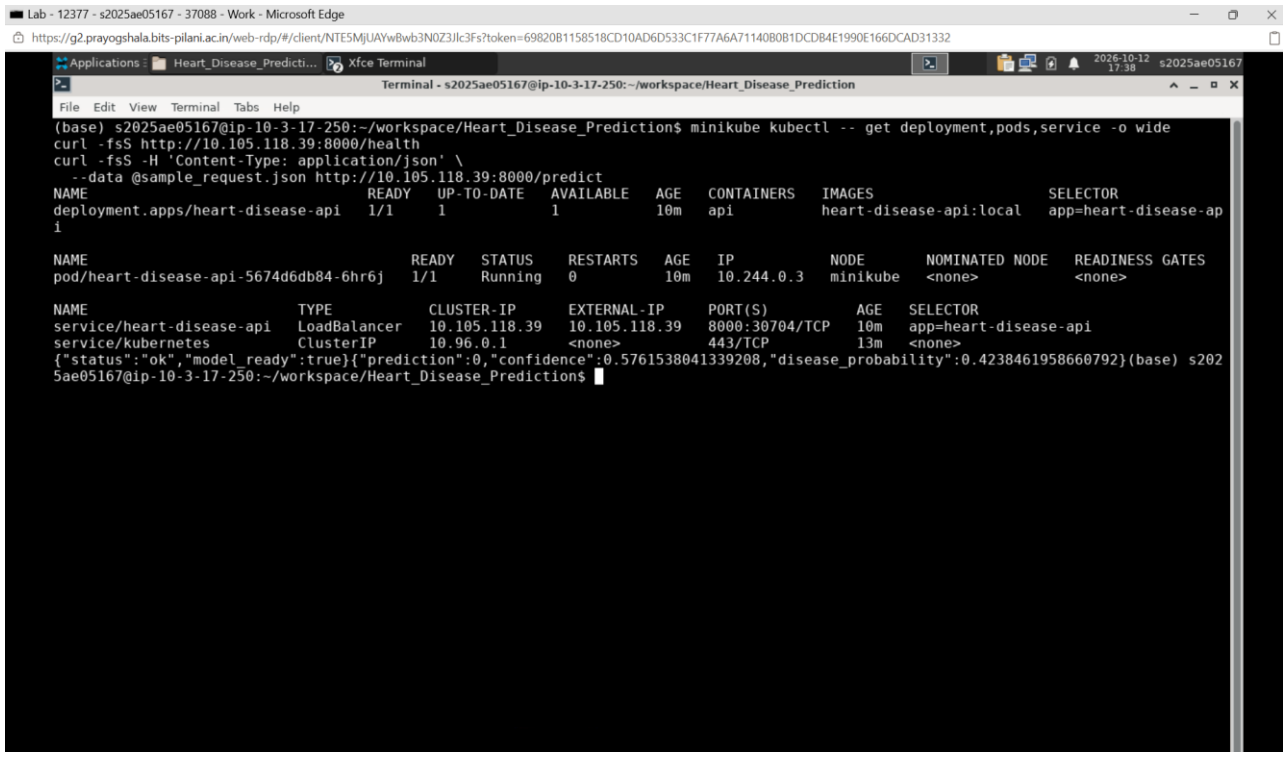


Figure A2. Minikube Deployment Ready 1/1, LoadBalancer external IP, and successful health and prediction responses through the service.